

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 3 Управляющие конструкции языка Julia

3.1 Условные операторы

3.1.1 Условный (тройной) оператор

Условный оператор вида **условие ? выражение_если_истина : выражение_если_ложь** выполняет второе выражение, если условие истинно, и третье – если ложно, при этом символы **?** и **:** обязательно отделяются пробелами.

```
In [9]: x = 0
        x > 0 ? println("x > 0") : println("x <= 0")
x <= 0
```

```
In [8]: x = -5
        x > 0 ? println("x > 0") : x == 0 ? println("x = 0") : println("x < 0")
x < 0
```

3.1.2 Условный оператор if

В Julia простейшая форма условного оператора имеет вид:

```
if логическое_выражение
    операторы, выполняемые, когда логическое выражение истинно
end
```

В такой форме действия после двоеточия выполняются, если логическое выражение истинно. Если же оно ложно, программа ничего не делает и

переходит к следующему оператору. Полная форма оператора **if**:

```
if логическое_выражение
    операторы, выполняемые, когда логическое выражение истинно
else
    операторы, выполняемые, когда логическое выражение ложно
end
```

Если нужно последовательно проверить несколько условий, используется расширенная форма с дополнительным оператором **elseif**:

```
if логическое_выражение
    операторы, выполняемые, когда логическое выражение истинно
elseif второе_логическое_выражение
    операторы, выполняемые, когда второе логическое выражение
истинно
elseif третье_логическое_выражение
    операторы, выполняемые, когда третье логическое выражение
истинно
...
else
    операторы, выполняемые, когда все логические выражения
ложны
end
```

Дополнительных условий и связанных с ними блоков **elseif** может быть сколько угодно. Если некоторое условие оказалось истинным, соответствующий блок кода выполняется, и дальнейшие условия не проверяются.

3.1.3 Примеры задач

Задача 1. Решить квадратное уравнение общего вида:

$$a \cdot x^2 + b \cdot x + c = 0$$

Версия 1.

```
In [8]: println("Введите коэффициент a")
a = parse(Float64, readline())
println("a = ", a)

println("Введите коэффициент b")
b = parse(Float64, readline())
println("b = ", b)

println("Введите коэффициент c")
c = parse(Float64, readline())
println("c = ", c)

d = b * b - 4 * a * c
x1 = (-b + sqrt(d)) / 2 / a
x2 = (-b - sqrt(d)) / 2 / a
```

```
println("Два действительных корня:")
print("x1 = ", x1, "\t", "x2 = ", x2)
```

Введите коэффициент a
a = -5.0
Введите коэффициент b
b = 3.0
Введите коэффициент c
c = 2.0
Два действительных корня:
x1 = -0.4 x2 = 1.0

Версия 2.

```
In [9]: println("Введите коэффициент a:")
a = parse(Float64, readline())
println("a = ", a)

println("Введите коэффициент b:")
b = parse(Float64, readline())
println("b = ", b)

println("Введите коэффициент c:")
c = parse(Float64, readline())
println("c = ", c)

d = b * b - 4 * a * c

if d >= 0
    x1 = (-b + sqrt(d)) / 2 / a
    x2 = (-b - sqrt(d)) / 2 / a

    println("Два действительных корня:")
    print("x1 = ", x1, "\t", "x2 = ", x2)
else
    print("Действительных корней нет")
end
```

Введите коэффициент a:
a = 4.0
Введите коэффициент b:
b = 5.0
Введите коэффициент c:
c = 6.0
Действительных корней нет

Версия 3. Исправим ошибку вычисления корня из отрицательного числа.

```
In [10]: println("Введите коэффициент a:")
a = parse(Float64, readline())
println("a = ", a)

println("Введите коэффициент b:")
b = parse(Float64, readline())
println("b = ", b)
```

```

println("Введите коэффициент c:")
c = parse(Float64, readline())
println("c = ", c)

# Вычисление дискриминанта
D = b^2 - 4*a*c

if D > 0
    # Два действительных корня
    x1 = (-b + sqrt(D)) / (2*a)
    x2 = (-b - sqrt(D)) / (2*a)

    println("Два действительных корня:")
    println("x_1 = ", x1)
    println("x_2 = ", x2)
elseif D == 0
    # Один действительный корень
    x = -b / (2*a)

    println("Один действительный корень:")
    println("x = ", x)
else
    # Два комплексных корня
    real = -b / (2*a)
    imag = sqrt(-D) / (2*a)

    println("Два комплексных корня:")
    println("x1 = ", real, " + ", imag, "i")
    println("x2 = ", real, " - ", imag, "i")
end

```

Введите коэффициент a:

a = 4.0

Введите коэффициент b:

b = 5.0

Введите коэффициент c:

c = 6.0

Два комплексных корня:

x1 = -0.625 + 1.0532687216470449i

x2 = -0.625 - 1.0532687216470449i

Задача 2. Решить кубическое уравнение общего вида:

$$a \cdot x^3 + b \cdot x^2 + c \cdot x + d = 0$$

Для решения используем приведение к каноническому виду и формулы Кардано.

1. Приведение к уравнению вида ($x^3 + px + q = 0$):

$$r = \frac{b}{a}, \quad s = \frac{c}{a}, \quad t = \frac{d}{a}$$

$$p = s - \frac{r^2}{3}, \quad q = \frac{2r^3}{27} - \frac{rs}{3} + t$$

2. Дискриминант:

$$D = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3$$

3. Решение в зависимости от знака дискриминанта:

D > 0 (один действительный и два комплексно-сопряжённых корня)

$$u = \sqrt[3]{-q/2 + \sqrt{D}}, \quad v = \sqrt[3]{-q/2 - \sqrt{D}}$$

$$x_1 = u + v - r/3$$

$$x_{2,3} = -\frac{u+v}{2} - r/3 \pm i\frac{\sqrt{3}}{2}(u-v)$$

D = 0 (все корни действительные, есть кратный)

$$x_1 = 2\sqrt[3]{-q/2} - r/3, \quad x_2 = x_3 = -\sqrt[3]{-q/2} - r/3$$

D < 0 (три действительных корня)

$$\phi = \arccos\left(-\frac{q}{2}\sqrt{-\frac{27}{p^3}}\right), \quad R = 2\sqrt{-p/3}$$

$$x_1 = R \cos(\phi/3) - r/3, \quad x_2 = R \cos((\phi + 2\pi)/3) - r/3, \quad x_3 = R \cos((\phi + 4\pi)/3) - r/3$$

```
In [12]: println("Введите коэффициент a")
a = parse(Float64, readline())
println("a = ", a)

println("Введите коэффициент b")
b = parse(Float64, readline())
println("b = ", b)

println("Введите коэффициент c")
c = parse(Float64, readline())
println("c = ", c)

println("Введите коэффициент d")
d = parse(Float64, readline())
println("d = ", d)

r = b / a
s = c / a
t = d / a
```

```

p = s - r^2/3
q = (2*r^3)/27 - (r*s)/3 + t
D = (q/2)^2 + (p/3)^3

if D > 0
    # Один действительный корень
    u = cbrt(-q/2 + sqrt(D)) #cbrt - вычисление кубического корня
    v = cbrt(-q/2 - sqrt(D))
    x1 = u + v - r/3
    println("Один действительный корень:")
    println("x1 = ", x1)

    # Два комплексно-сопряженных корня
    real = -(u + v)/2 - r/3
    imag = abs(u - v)*sqrt(3)/2
    println("Два комплексных корня:")
    println("x2 = ", real, " + ", imag, "im")
    println("x3 = ", real, " - ", imag, "im")

elseif D == 0
    # Кратные корни
    x1 = 2*cbrt(-q/2) - r/3
    x2 = -cbrt(-q/2) - r/3
    println("Два действительных корня (один кратный):")
    println("x1 = ", x1)
    println("x2 = ", x2, " (кратный)")

else
    # Три действительных корня
    fi = acos(-q/2 * sqrt(-27/p^3))
    root = 2 * sqrt(-p/3)

    x1 = root * cos(fi/3) - r/3
    x2 = root * cos((fi + 2π)/3) - r/3
    x3 = root * cos((fi + 4π)/3) - r/3

    println("Три действительных корня:")
    println("x1 = ", x1)
    println("x2 = ", x2)
    println("x3 = ", x3)

end

```

Введите коэффициент a

a = 4.0

Введите коэффициент b

b = 5.0

Введите коэффициент c

c = 8.0

Введите коэффициент d

d = -6.0

Один действительный корень:

x1 = 0.5154597608712708

Два комплексных корня:

x2 = -0.8827298804356354 + 1.459729914865296im

x3 = -0.8827298804356354 - 1.459729914865296im

3.2 Операторы цикла Julia

3.2.1 Цикл while

Общая структура цикла **while**

```
while условие
    оператор 1
    ...
    оператор n
end
```

Задача 1. Напишите программу, которая ищет и выводит наибольшее отрицательное число из всех введенных пользователем значений, ввод прекращается, когда вводится 0.

```
In [14]: # Запрашиваем у пользователя первое число
println("Введите N ")
N = parse(Float64, readline())
println("N = ", N)

# Инициализируем счетчик отрицательных чисел и переменную для хранения максим
kp = 0      # счетчик отрицательных чисел
mx = 0      # будет хранить максимальное отрицательное число

# Основной цикл, который работает пока пользователь не введет 0
while N != 0
    # Проверяем, является ли введенное число отрицательным
    if N < 0
        # Увеличиваем счетчик отрицательных чисел
        kp = kp + 1

        # Если это первое отрицательное число, сразу запоминаем его как макс
        if kp == 1
            mx = N
        # Иначе сравниваем с текущим максимальным и обновляем при необходим
        elseif N > mx
            mx = N
        end
    end

    # Запрашиваем следующее число у пользователя
    println("Введите N ")
    N = parse(Float64, readline())
    println("N = ", N)
end

# После завершения цикла выводим максимальное отрицательное число
println("Количество отрицательных чисел: ", kp)
println("Максимальное отрицательное число: ", mx)
```

Введите N

```

N = 8.0
Введите N
N = -7.0
Введите N
N = -345.0
Введите N
N = -7.0
Введите N
N = 6.0
Введите N
N = 8.0
Введите N
N = -35.0
Введите N
N = -1.0
Введите N
N = 0.0
Количество отрицательных чисел: 5
Максимальное отрицательное число: -1.0

```

3.2.2 Оператор for

Цикл for можно организовать так: `i1` – начальное значение, `i2` – конечное, а `i3` (по умолчанию равный 1) – шаг, который можно не указывать, если он равен единице.

```

for i in i1:i2
    оператор 1
    ...
    оператор n
end
for i = i1:i2
    оператор 1
    ...
    оператор n
end
for i in i1:i3:i2
    оператор 1
    ...
    оператор n
end
for i = i1:i3:i2
    оператор 1
    ...
    оператор n
end

```

Задача 1. Переменная x меняется от x_1 до x_k с шагом dx . Значение y вычисляется по формуле

$$e^{\sin(x)} \cos(x)$$

Найти сумму и произведение значений y , минимальное и максимальное значение y .

```
In [15]: println("Введите начальное значение xn")
xn = parse(Float64, readline())
println("xn = ", xn)

println("Введите конечное значение xk")
xk = parse(Float64, readline())
println("xk = ", xk)

println("Введите шаг dx")
dx = parse(Float64, readline())
println("dx = ", dx)

# Инициализация переменных
s = 0.0 # Сумма значений y
p = 1.0 # Произведение значений y
min = Inf # Минимальное значение y
max = -Inf # Максимальное значение y

for x in xn:dx:xk
    y = exp(sin(x)) * cos(x)

    println("x=$x \t y=$y") # Вывод текущих значений

    s += y
    p *= y

    if y < min
        min = y
    end
    if y > max
        max = y
    end
end
println("Сумма=$s\nПроизведение=$p\nМинимум=$min\nМаксимум=$max")
```

```
Введите начальное значение xn
xn = 4.0
Введите конечное значение xk
xk = 5.0
Введите шаг dx
```

```

dx = 0.1
x=4.0    y=-0.3066661772342105
x=4.1    y=-0.25360722271836467
x=4.2    y=-0.20507212889574455
x=4.3    y=-0.16033967719350098
x=4.4    y=-0.11866796146370862
x=4.5    y=-0.07930964690547916
x=4.6    y=-0.0415197321954094
x=4.7    y=-0.004557884361316417
x=4.8    y=0.03231277189004458
x=4.9    y=0.06982868939983652
x=5.0    y=0.10872913262953789
Сумма=-0.9588698370483153
Произведение=1.1173944655306317e-12
Минимум=-0.3066661772342105
Максимум=0.10872913262953789

```

Операторы управления циклом. Оператор **continue** начинает следующий проход цикла, минуя оставшееся тело цикла (**for** или **while**). Оператор **break** досрочно прерывает цикл.

```

In [45]: i = 0
while i < 10
    i += 1 # увеличиваем i на 1

    if i % 2 == 0
        continue # пропускаем чётные числа
    end

    println("Нечётное i = ", i)

    if i >= 7
        break # выходим из цикла, если i >= 7
    end
end

```

```

Нечётное i = 1
Нечётное i = 3
Нечётное i = 5
Нечётное i = 7

```

3.3 Исключения

try/catch – это блок для обработки исключений.

```

try
    # код, который может вызвать ошибку
catch
    # код, выполняемый при возникновении ошибки
end

```

В блоке **catch** можно указать переменную, например **catch n**. Эта переменная будет содержать информацию об ошибке.

```

try
    # код, который может вызвать ошибку
catch n
    # код, выполняемый при возникновении ошибки
    # переменная n содержит информацию об ошибке
end

```

Пример 1. Ошибка выхода за пределы массива

```

In [50]: # Массив с числами
arr = [1, 2, 3]

try
    # Попытка обратиться к несуществующему индексу массива
    println(arr[5])
catch
    println("Ошибка: выход за пределы массива.")
end

```

Ошибка: выход за пределы массива.

Пример 2. Обработка ошибки преобразования типов.

```

In [52]: try
    # Попытка преобразовать строку в число
    num = parse{Int, "abc"} # Эта строка вызовет ошибку
    println(num)
catch n
    # Этот блок выполнится при возникновении ошибки
    println("Ошибка: не удалось преобразовать строку в число.")
    println("Тип ошибки: ", typeof(n))
    println("Сообщение: ", n.msg)
end

```

Ошибка: не удалось преобразовать строку в число.

Тип ошибки: ArgumentError

Сообщение: invalid base 10 digit 'a' in "abc"

Пример 3. Работа с файлами (например, файл не существует).

```

In [53]: try
    # Попытка открыть несуществующий файл
    file = open("file.txt", "r")
    println(readline(file))
catch
    println("Ошибка: файл не найден.")
end

```

Ошибка: файл не найден.

Подробнее о работе с файлами в Julia, включая создание, чтение и запись вы можете найти в **Главе 11**.